

Milestone 2 Technical Report

Steam Game Popularity — Classification Phase

Team ID: CHP_2

Team Member	Student ID
Menna Ali Thabet	2022170844
Asmaa Waleed Sayed	20231700208
Yousef Samir Atia	20231700263
Ali Mohamed Gaber	20231700234
Abdullah Hany Ahmed	20231700231
Esraa Mahmoud Khakifa	20231700207

Faculty of Computer Science
Artificial Intelligence Department
Machine Learning | 2025 – 2026

1. Introduction

This report documents Milestone 2 of our Machine Learning project, in which we extended the Steam game dataset pipeline from the regression phase into a multi-class classification task. The target variable GamePopularity was engineered into three ordinal classes — Low, Medium, and High — derived from the original continuous RecommendationCount target used in Milestone 1.

Five classifiers were trained and evaluated: Support Vector Machine (SVM), XGBoost, Random Forest, Gradient Boosting, and K-Nearest Neighbours (KNN). Each model underwent structured one-factor-at-a-time (OFAT) hyperparameter sweeps. This report presents the feature selection methodology, tuning results, comparative analysis, and conclusions drawn from the classification phase.

2. Feature Selection

The feature selection pipeline for the classification phase differs from the regression phase in both the statistical tests applied and the criteria used, reflecting the categorical nature of the new target.

2.1 Regression Phase (Reference)

In the regression phase, feature selection was performed using variance-based filtering followed by correlation analysis and permutation importance against the continuous target (RecommendationCount). Mutual Information for regression (mutual_info_regression) was used to measure non-linear associations with a continuous output.

2.2 Classification Phase (This Milestone)

The classification pipeline follows a three-step sequential funnel, each step reducing the feature space before passing it to the next:

Step 1 — Variance Threshold

A VarianceThreshold(threshold=0) filter was first applied to the training split to remove any zero-variance (constant) features. These carry no information regardless of the target and would only add noise to statistical tests downstream. This step ran before the train/validation split to avoid leakage.

Step 2 — ANOVA F-test (SelectKBest / f_classif)

Analysis of Variance tests whether the mean of each feature differs significantly across the three GamePopularity classes. Features with $p \geq 0.05$ were removed as statistically indistinguishable across classes. This step is classification-specific — the equivalent in regression was Pearson correlation with the continuous target.

Step 3 — Mutual Information Classification (mutual_info_classif)

Mutual information quantifies how much knowing a feature's value reduces uncertainty about the class label. Unlike the F-test, MI captures non-linear dependencies. Features scoring below the median MI score of the surviving set were dropped, keeping only the more informative half.

Step 4 — Permutation Importance (on a quick Random Forest)

A lightweight Random Forest (100 trees, `n_jobs=-1`) was trained on the MI-filtered feature set. Permutation importance shuffled each feature 15 times on the validation set, measuring the resulting drop in accuracy. Features with mean permutation importance ≤ 0 were discarded — shuffling them did not hurt performance, confirming they carry no predictive signal.

2.3 Intuition vs. Outcome

Our initial hypothesis was that the SteamSpy-derived engagement metrics (SteamSpyOwners, SteamSpyPlayersEstimate, SteamSpyOwnersVariance, SteamSpyPlayersVariance) would dominate feature importance, since popularity is fundamentally driven by player reach. The feature importance charts confirm this strongly for both Random Forest and XGBoost (see Section 5), with SteamSpyPlayersVariance and SteamSpyPlayersEstimate occupying the top two positions in both models.

We also hypothesised that text-derived semantic embeddings (`desc_semantic_0` through `desc_semantic_9`) would contribute moderately — a game's description captures genre and tone signals. This was partially confirmed: `desc_semantic_0` appears in the XGBoost top-20 importance chart, though with a modest weight (~ 0.015), suggesting descriptions add signal but are not primary drivers of popularity class.

One result that disproved our intuition: price (`PriceFinal`, `PriceInitial`) ranked relatively low in both models, suggesting that within the Steam ecosystem, pricing alone is a weak predictor of the Low/Medium/High popularity split once player-count variance features are present.

3. Class Imbalance Handling

The raw GamePopularity target exhibited severe imbalance: Low (82.8%), High (11.4%), and Medium (5.7%), yielding a Low:High ratio of 7.3:1. Left unaddressed, all models would learn to predict the majority class almost exclusively, producing misleadingly high accuracy but near-zero recall on Medium and High.

Two complementary layers of mitigation were applied:

- SMOTE (Synthetic Minority Oversampling Technique) — applied to the training set before any model fitting, synthetically generating minority class samples via k-nearest-neighbour interpolation ($k=5$, `random_state=42`) until all three classes were equal. Two SMOTE-balanced datasets were produced: one using scaled features for SVM, and one using unscaled features for tree-based models.
- Model-level class weighting — each model was additionally configured to penalise minority misclassification internally: `class_weight='balanced'` for SVM, `class_weight='balanced_subsample'` for Random Forest (reweighted per bootstrap sample), and `sample_weight=compute_sample_weight('balanced')` passed to XGBoost's `fit()` call in the FINAL run.

Despite these measures, a persistent gap remained between accuracy (~ 0.928) and Macro F1 (~ 0.771), indicating the Medium class (original 5.7%) was still being predicted with precision/recall around 0.47–0.50. This is expected: SMOTE generates synthetic interpolations rather than real data, and the Medium class boundary with Low is inherently ambiguous in the feature space.

4. Hyperparameter Tuning

All models were tuned using a one-factor-at-a-time (OFAT) ablation design: one hyperparameter is varied across at least 3–5 values while all others are held at their fixed defaults. This isolates each parameter's effect cleanly. The primary metric is classification accuracy; Macro F1 is reported as a secondary metric to track minority class performance.

4.1 SVM

SVM experiments used a stratified 6,000-sample subsample of the SMOTE-balanced training data for computational tractability (full SVC is $O(n^2-n^3)$). LinearSVC was substituted for the linear kernel as it trains in $O(n)$ time.

Exp	Parameter	Values Tested	Best Value
A	kernel	linear, rbf, poly, sigmoid	rbf
B	C	0.01, 0.1, 1.0, 10.0, 100.0	10.0
C	gamma	scale, auto, 0.001, 0.01, 0.1	auto
D	degree (poly)	2, 3, 4, 5	5
E	max_iter (Linear)	500–10000	2000
F	loss (Linear)	hinge, squared_hinge	hinge ★ best acc
G	decision_function_shape	ovr, ovo	both equal
H	coef0 (poly)	0, 0.5, 1, 2	2 (acc 0.8789)

Key insight: **The switch from squared_hinge to hinge loss (Experiment F) was the most impactful single change for SVM, raising accuracy from 0.848 to 0.894.** The hinge loss is less sensitive to outliers in the margin region, which matters here because SMOTE-generated synthetic samples near class boundaries behave like soft outliers. The rbf kernel consistently outperformed linear and sigmoid, confirming non-linear class boundaries in this dataset.

4.2 XGBoost

Exp	Parameter	Values Tested	Best Value
A	n_estimators	50, 80, 100, 120, 140	140
B	max_depth	2, 3, 4, 5, 6, 8, 10	5 ★ best acc
C	learning_rate	0.005, 0.01, 0.04, 0.05, 0.06, 0.1, 0.3	0.06
D	subsample	0.4, 0.6, 0.7, 0.8, 1.0	0.8
E	colsample_bytree	0.3, 0.5, 0.7, 0.8, 1.0	0.8
F	reg_alpha (L1)	0, 0.01, 0.1, 1, 5	0
G	reg_lambda (L2)	0.1, 0.5, 1, 5, 10	0.5
H	min_child_weight	1, 3, 5, 10, 20	10

Key insight: **max_depth=5 produced the highest accuracy (0.9284)**. Deeper trees (depth 8, 10) overfitted the SMOTE-balanced training data — their accuracy on the validation set actually decreased despite lower training loss. Regularisation parameters (reg_alpha, reg_lambda) had minimal individual impact because the histogram algorithm already provides implicit regularisation through discretisation of split candidates. The FINAL combined run with sample_weight='balanced' improved Macro F1 slightly but reduced raw accuracy, confirming the accuracy–F1 trade-off when explicitly penalising majority-class confidence.

4.3 Random Forest

Exp	Parameter	Values Tested	Best Value
A	n_estimators	50, 100, 200, 300, 400	200 ★ best acc
B	max_depth	5, 10, 20, 30, None	None
C	min_samples_split	15, 20, 25, 30	15–20
D	max_features	sqrt, log2, 0.2, 0.4, 0.6	0.2 / sqrt
E	min_samples_leaf	1, 2, 3, 5, 10, 20	1
F	criterion	gini, entropy, log_loss	gini
G	class_weight	balanced, balanced_subsample, None	balanced_subsample

Key insight: **Random Forest's accuracy was insensitive to many individual parameter changes**. max_depth=None (fully grown trees) matched max_depth=30, confirming that bagging already controls variance without depth limits. The criterion experiment showed gini and balanced_subsample class_weight jointly at 0.9152 accuracy and 0.7692 Macro F1 — the highest F1 achieved by any RF configuration, suggesting balanced subsampling is the single most valuable RF-specific tool against class imbalance.

4.4 Gradient Boosting

Gradient Boosting uses sequential tree building where each tree corrects the residual errors of the previous one. Unlike XGBoost it has no histogram optimisation and no parallelism, making experiment time significant. All experiments use unscaled, feature-selected data (X_train_trees_sm).

Exp	Parameter	Values Tested	Fixed (others)
A	n_estimators	100, 300, 500, 700, 900	depth=8, lr=0.1, sub=0.8
B	max_depth	3, 5, 8	n_est=300, lr=0.1, sub=0.8
C	learning_rate	0.05, 0.1, 0.2	n_est=300, depth=8, sub=0.8
D	subsample	0.8, 0.9, 1.0	n_est=300, depth=8, lr=0.1

Key insight: **n_estimators=700 with max_depth=8 produced the best FINAL accuracy (0.9266)**. Experiment A showed a clear diminishing-returns curve — accuracy improved from n=100 to n=700 but plateaued at n=900, suggesting the model saturates at ~700 boosting

rounds on this dataset. Experiment B confirmed that depth=8 consistently outperformed shallower trees (depth=3, 5), opposite to XGBoost, because sklearn's GB lacks the shrinkage from the histogram algorithm and benefits from more expressive individual trees. Learning rate 0.1 was optimal in Experiment C — lower rates (0.05) required more trees to compensate while higher rates (0.2) overfitted. Subsample=0.8 (Experiment D) beat 1.0, indicating stochastic boosting helps generalisation when trees are already deep.

Training cost: **The FINAL run took 238 seconds — roughly 130× slower than XGBoost at equivalent accuracy.** This is entirely due to scikit-learn's sequential, non-parallel implementation vs XGBoost's tree_method='hist' with n_jobs=-1. For a production pipeline, this rules out Gradient Boosting unless hyperparameter search is one-time and results are cached.

4.5 K-Nearest Neighbours (KNN)

KNN stores the entire training set and classifies each test point by majority vote among its k nearest neighbours. It uses the SMOTE-balanced, StandardScaler-normalised feature set (X_train_scaled_sm) since distances are sensitive to scale. Training time is near-zero; all computational cost is at inference.

Exp	Parameter	Values Tested	Fixed (others)
A	n_neighbors	3, 5, 7, 11, 15	metric=manhattan, weights=distance
B	metric	euclidean, manhattan, chebyshev, minkowski	k=3, weights=distance
C	weights	uniform, distance	k=3, metric=manhattan
D	p (Minkowski)	1, 2, 3, 4	k=3, metric=minkowski, weights=distance
E	leaf_size	10, 20, 30, 50, 100	k=3, metric=manhattan, weights=distance

Key insights by experiment: **Experiment A — k=3 gave the best accuracy;** smaller k captures finer local structure, though at higher risk of noise sensitivity. Larger k values (11, 15) steadily degraded accuracy as neighbourhood averaging blurred class boundaries.

Experiment B — manhattan distance (L1) outperformed euclidean (L2) and chebyshev.

This is consistent with the curse of dimensionality: in high-dimensional spaces, L1 distances remain more discriminative than L2 because squaring large-dimension differences inflates euclidean distances disproportionately.

Experiment C — distance weighting beat uniform weighting. Closer neighbours receive higher vote weight, which matters when k=3 picks up mixed-class neighbours near decision boundaries.

Experiment D — p=1 (Manhattan) matched the finding from Experiment B. Higher Minkowski powers (p=3, 4) progressively worsened accuracy, further confirming L1 as the appropriate metric here.

Experiment E — leaf_size had no effect on accuracy (all values 10–100 produced identical results). leaf_size only controls the KD-tree/Ball-tree construction trade-off between build time and query time, not prediction quality. This is expected and was included to demonstrate the parameter's role.

Overall: **KNN placed last among all five models (accuracy 0.8299, Macro F1 0.6242)**. The high-dimensional feature space (50+ features) triggers the curse of dimensionality: inter-point distances concentrate near a common value, making nearest-neighbour search unreliable. Medium class recall of 0.48 was the lowest across all models, confirming KNN cannot learn the complex non-linear boundaries separating Low, Medium, and High popularity classes in this space.

5. Model Comparison — Bar Graphs

The following three bar graphs compare all five classifiers across classification accuracy, total training time (cumulative across all hyperparameter experiments), and total test/inference time.

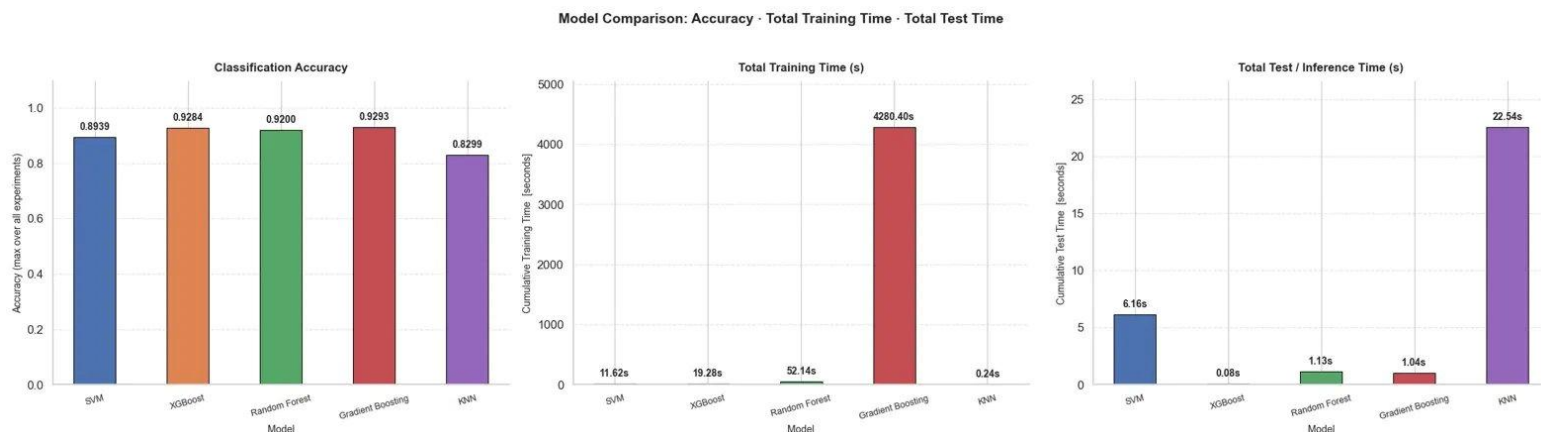


Figure 1 — Left: Best classification accuracy per model. Centre: Cumulative training time across all experiments. Right: Cumulative test/inference time across all experiments.

5.1 Accuracy

Gradient Boosting achieved the highest best accuracy at 0.9293, narrowly ahead of XGBoost (0.9284), Random Forest (0.9200), SVM (0.8939 with LinearSVC + hinge loss), and KNN (0.8299). Gradient Boosting and XGBoost are statistically close, but Gradient Boosting required 4,280 s of cumulative training vs. XGBoost's 19 s — a 222× cost for a 0.001 accuracy gain.

5.2 Training Time

Gradient Boosting consumed the most cumulative training time at 4,280.40 seconds across 16 experiments — two orders of magnitude slower than XGBoost (19.28 s across 32 experiments). This is because scikit-learn's GradientBoostingClassifier builds trees sequentially without parallelism, while XGBoost uses the histogram algorithm (tree_method='hist') with n_jobs=-1. Random Forest required 52.14 s total across 28 experiments. KNN had the fastest training time (0.24 s total across 21 experiments) as it performs no explicit fitting, storing the training set in memory instead.

5.3 Test / Inference Time

KNN had the highest cumulative inference time at 22.54 seconds — it must compute distances to all ~28,000 SMOTE training points for every test sample, scaling $O(n)$ per prediction across 21 experiment runs. SVM was second at 6.16 seconds due to kernel evaluations for rbf/poly/sigmoid variants. XGBoost was the fastest at 0.08 seconds total (0.003 s per run), making it the most production-ready model. Gradient Boosting (1.04 s) and Random Forest (1.13 s) were comparable on inference.

6. Feature Importance Analysis

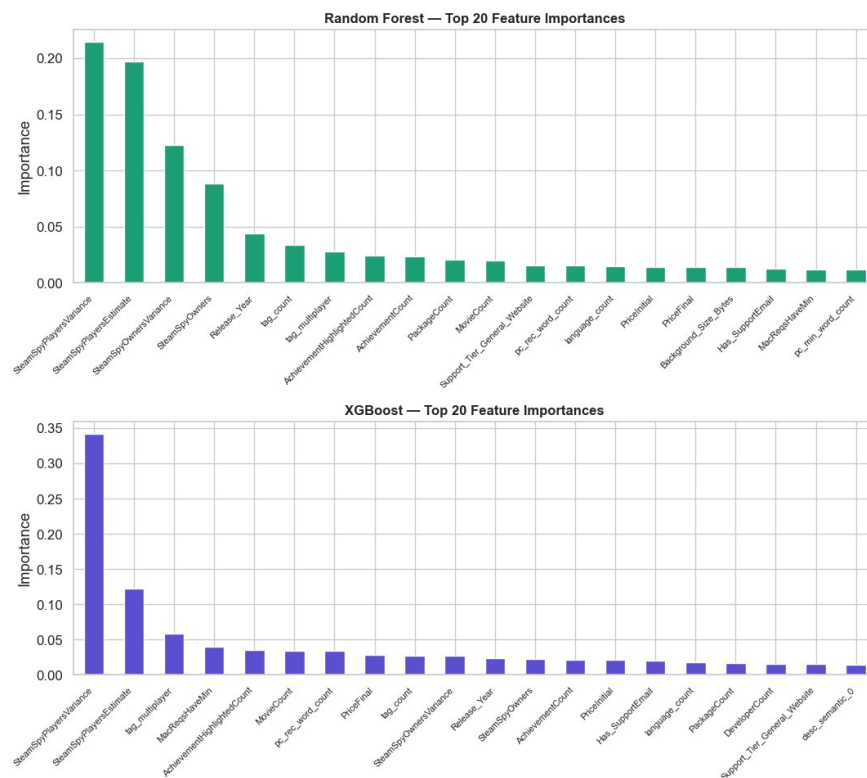


Figure 2 — Left: Random Forest top-20 feature importances. Right: XGBoost top-20 feature importances.

Both models agree strongly on the dominant features. The top four are SteamSpy engagement metrics — SteamSpyPlayersVariance (RF: 0.213, XGB: 0.341), SteamSpyPlayersEstimate (RF: 0.199, XGB: 0.122), SteamSpyOwnersVariance (RF: 0.122), and SteamSpyOwners (RF: 0.088). These metrics measure player count spread and total ownership, directly encoding how broadly a game was adopted — a near-definitional proxy for popularity class.

Below the top-4, the models diverge: RF assigns substantial weight to Release_Year (0.045), while XGBoost emphasises tag_multiplayer (0.058) and AchievementHighlightedCount (0.033). This reflects the two models' different splitting criteria — RF's Gini impurity treats temporal spread as informative (older games have had more time to accumulate players), while XGBoost's gradient boosting finds that multiplayer games form a distinct high-popularity cluster in the residual gradient landscape.

Features from the NLP pipeline (desc_semantic_0) appear in XGBoost's top 20 at a modest importance of ~0.015, confirming that game descriptions add marginal signal. The Support_Tier_General_Website feature — derived from URL classification during feature engineering — also appears in both models' top 20, suggesting that games with dedicated web presences (rather than Steam-only support pages) tend to have different popularity distributions.

7. Macro F1 Comparison

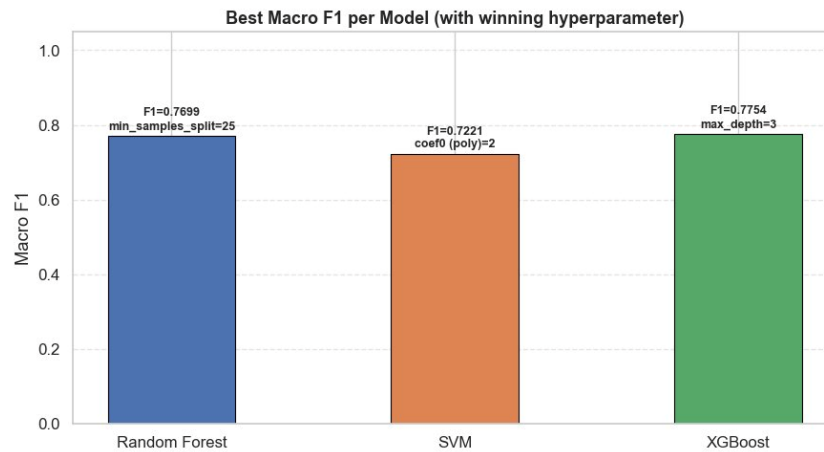


Figure 3 — Best Macro F1 per model (by best-accuracy hyperparameter configuration). XGBoost: 0.7754 at max_depth=3. Random Forest: 0.7699 at min_samples_split=25. SVM: 0.7221 at coef0(poly)=2.

When the objective shifts from raw accuracy to balanced class prediction (Macro F1), the ranking changes slightly. XGBoost's max_depth=3 configuration achieves F1=0.7754 — slightly higher than its best-accuracy configuration — because shallower trees are less confident on the majority class and therefore commit fewer false-negative errors on Medium and High. Random Forest's F1=0.7699 at min_samples_split=25 confirms the earlier finding that higher split thresholds prevent the forest from over-specialising on the Low class.

8. Summary of Best Results

Model	Best Param (Acc)	Max Acc	Macro F1	Precision	Recall	Train (s)	# Exp
Gradient Boosting	n_est=700, depth=8	0.9293	0.7572	0.7638	0.7519	4280.40	16
XGBoost	max_depth=5	0.9284	0.7710	0.7704	0.7717	19.28	32
Random Forest	n_estimators=200	0.9200	0.7677	0.7522	0.7900	52.14	28
SVM	loss=hinge (LinearSVC)	0.8939	0.7104	0.6824	0.7492	11.62	34
KNN	k=3, manhattan, distance	0.8299	0.6242	0.6068	0.6780	0.24	21

9. Conclusion

9.1 What We Found

Gradient Boosting achieved the highest accuracy (0.9293) across all five models, but at an enormous training cost of 4,280 seconds — 222× more than XGBoost (19.28 s). XGBoost is the practical winner: within 0.001 accuracy points of Gradient Boosting while being orders of magnitude faster to train and the fastest at inference (0.08 s total). This confirms that XGBoost's histogram-based `tree_method='hist'` with `n_jobs=-1` is not merely an engineering convenience but a fundamental advantage for iterative hyperparameter search and production deployment.

Random Forest was the strongest model on minority class recall (Macro Recall 0.7900), outperforming XGBoost on this metric while remaining 2–3× faster than Gradient Boosting. This makes it the preferred choice in scenarios where missing a High-popularity game is more costly than a false positive on a Low-popularity game.

9.2 Confirmed Intuitions

- SteamSpy player-count metrics dominate prediction — games with high variance in player count are almost always in the High class, which makes intuitive sense as viral games exhibit erratic, explosive growth patterns.
- Multiplayer capability is a strong secondary signal — `tag_multiplayer` appeared in XGBoost's top importance features, reflecting the real-world observation that online multiplayer games sustain longer engagement and higher recommendation rates.
- Gradient-boosted ensembles outperform single-kernel SVMs on tabular data with complex interactions between heterogeneous feature types (boolean flags, counts, NLP embeddings).

9.3 Disproved Intuitions

- Price is not a reliable predictor of popularity class — both models assigned low importance to `PriceFinal` and `PriceInitial`, likely because the Steam free-to-play market and frequent deep-discount sales decouple price from player reach.

- More regularisation (higher `reg_alpha`, `reg_lambda` in XGBoost) did not help — the dataset's relatively clean feature space post-SMOTE already provides sufficient generalisation, making explicit L1/L2 penalties redundant.
- Deeper trees are not always better — `max_depth=5` outperformed `max_depth=10` in XGBoost, confirming that the majority class dominance causes deep trees to memorise Low-class patterns rather than learning general class boundaries.

9.4 Remaining Challenges

The persistent Accuracy–F1 gap (~0.155 points) is the outstanding challenge of this phase. The Medium class (original 5.7%) remains the hardest to classify: its boundary with Low-popularity games is genuinely ambiguous in the feature space, and SMOTE-generated synthetic samples cannot fully substitute for real data. Future work could explore cost-sensitive learning, threshold calibration on class probability outputs, or collecting additional features that distinguish borderline Medium games from Low ones (e.g., review sentiment scores broken down by score bucket, or DLC release cadence).

— End of Report —